

- CS 423 / 523 - Color Segmentation & Object Counting: Detaylı Proje Rehberi
 - 1. Projenin Genel Amacı
 - 2. Kullanılan Teknolojiler
 - 3. Dosya ve Klasör Yapısı
 - 4. Kodun Genel Çalışma Akışı
 - 5. Algoritma Mantığı (Adım Adım)
 - 6. Önemli Fonksiyonlar / Sınıflar
 - 7. Veri Akışı Örneği
 - 8. Sunum İçin Anlatım Rehberi
 - 9. Muhtemel Soru-Cevaplar
 - 10. Güçlü ve Zayıf Yönler
 - 11. Basit Özet (Hiç Bilmeyene Anlatım)

CS 423 / 523 - Color Segmentation & Object Counting: Detaylı Proje Rehberi

Bu rehber, projeyi baştan sona anlayabilmen, kodun nasıl çalıştığını içselleştirebilmen ve jüri karşısında kendine güvenerek sunabilmen için özel olarak, "hiç bilmeyen birine anlatır gibi" hazırlanmıştır.

1. Projenin Genel Amacı

Bu proje ne yapıyor? Bu proje, bilgisayarlı görü (Computer Vision) tekniklerini kullanarak bir fotoğrafın içindeki belirli renkteki (Kırmızı, Mavi, Yeşil, Sarı) nesneleri bulan, bunları arka plandan ayıran (segmentasyon) ve kaç adet nesne olduğunu sayan bir yazılım sistemidir.

Hangi problemi çözüyor? İnsanların gözle saymakta zorlanacağı veya çok zaman kaybedeceği nesnelerin (örneğin üretim bandındaki hatalı renkli ürünler, mikroskop altındaki hücreler veya otoyoldaki kırmızı arabalar) otomatik olarak sayılmasını sağlar. Otomasyon ve yapay görme problemlerine temel bir çözüm sunar.

Kullanıcı açısından nasıl çalışıyor? Kullanıcı sisteme bir klasör dolusu fotoğraf ve bu fotoğraflarda "hangi renkten kaç tane nesne olması gerektiğini" yazan bir JSON (ayar) dosyası verir. Program çalışır ve her bir fotoğraf için:

- Nesneleri bulur,
 - Arka planı siyaha boyar (Maskeleme),
 - Bulduğu nesneleri sayar,
 - Kullanıcının beklediği sayıyla, programın bulduğu sayıyı karşılaştırıp bir başarı puanı (% Doğruluk) ve rapor üretir.
-

2. Kullanılan Teknolojiler

Projenin en büyük özelliği, piyasada hazır bulunan devasa kütüphaneleri (örneğin OpenCV) kullanmak yerine, temel algoritmaların "sıfırdan" (from scratch) yazılmış olmasıdır. Bu, algoritma bilgisini kanıtlamak için mükemmel bir detaydır.

- **Python:** Projenin yazıldığı ana programlama dili.
 - **NumPy:** Python'un en güçlü matematik ve matris kütüphanesi. Resimleri piksellerden oluşan devasa matrisler olarak ele alıp üzerlerinde çok hızlı matematiksel işlemler (örneğin "Değeri 100'den büyük olan pikselleri 1 yap") yapmak için kullanıldı. *Projeyi sırtlayan asıl kütüphanedir.*
 - **Pillow (PIL):** Görüntüleri bilgisayardan okumak (açmak) ve işlendikten sonra tekrar bilgisayara **.png** olarak kaydetmek için kullanılan giriş/çıkış (I/O) kütüphanesi.
 - **Pytest:** Yazdığımız kodların doğru çalışıp çalışmadığını otomatik olarak test eden kütüphane.
 - **Makefile:** Terminalde uzun uzun kod yazmak yerine sadece **make build-real-bundle** diyerek tüm projenin tek tıkla çalışmasını sağlayan bir kısayol aracı.
-

3. Dosya ve Klasör Yapısı

Proje oldukça düzenli ve profesyonel bir mimariye sahiptir.

- **data/ (Veri Klasörü):** Projenin kalbi burasıdır.
 - **data/real/raw/:** Bizim telefonla veya kamerayla çektiğimiz, işlenmek üzere bekleyen 300x300 çözünürlüğündeki gerçek test fotoğrafları burada durur.
 - **data/real/metadata/dataset.json:** Fotoğrafların "Cevap Anahtarı"dır. "real_red_1.png dosyasında 1 tane kırmızı nesne var" bilgisini tutar.
- **configs/ (Ayar Klasörü):**

- **multi-color-template.json**: Algoritmanın "Kırmızı nedir? Mavi nedir?" sorusuna cevap bulduğu yerdir. Hangi rengin hangi piksel aralıklarında olduğunu ve hangi hassasiyetle taranması gerektiğini belirtir.
- **src/cs423_segmentation/ (Kaynak Kod Klasörü)**: Tüm sihrin gerçekleştiği yer. Algoritmalarımızın bulunduğu ana kod deposudur.
- **results/ (Sonuç Klasörü)**: Program çalıştıktan sonra ürettiği maskelenmiş siyah-beyaz görüntüleri ve PDF/JSON formatındaki hata-başarı analiz raporlarını buraya kaydeder.

4. Kodun Genel Çalışma Akışı

Kullanıcı terminale **make build-real-bundle** yazdığında arka planda tam olarak şunlar olur:

1. **Başlangıç**: **cli.py** (Command Line Interface) dosyası tetiklenir. Kullanıcının hangi veri setini (**dataset.json**) kullanmak istediğini anlar.
2. **Ayarların Okunması**: Sistemin kırmızı, yeşil, mavi aralıklarını anlaması için **configs** klasöründeki profil ayarları yüklenir.
3. **Döngü Başlar**: Program **dataset.json** içindeki fotoğrafları tek tek açmaya başlar.
4. **Renk Uzayı Dönüşümü**: Açılan fotoğraf RGB (Kırmızı-Yeşil-Mavi) formatından HSV (Renk-Doygunluk-Parlaklık) formatına çevrilir (Çünkü HSV, ışık değişimlerine karşı çok daha dirençlidir).
5. **Eşikleme (Thresholding)**: Ayar dosyasındaki sınırlar kullanılarak "Sadece belirlediğim renkteki pikselleri BEYAZ, gerisini SİYAH yap" emri verilir.
6. **Temizlik (Morphology)**: Oluşan siyah-beyaz görüntüdeki ufak tefek pürüzler ve gölgeler (Gürültü/Noise) matematiksel olarak silinir.
7. **Sayım (Connected Components)**: Siyah zemin üzerindeki beyaz bölgeler sayılır.
8. **Karşılaştırma**: Bulunan sayı ile cevap anahtarındaki sayı karşılaştırılır.
9. **Kayıt**: İşlenen resimler ve hazırlanan rapor **results/** klasörüne kaydedilir. Sonraki resme geçilir.

5. Algoritma Mantığı (Adım Adım)

Projenin merkezinde sırayla işleyen 4 ana algoritma adımı vardır:

A. Renk Uzayı Dönüşümü (RGB -> HSV) Bilgisayarlar renkleri RGB (Red, Green, Blue) olarak görür. Ancak RGB ışığa çok duyarlıdır; gölgede kalan kırmızı bir elma, RGB'ye göre siyaha veya kahverengiye yakın görünür. Bu yüzden resmi **HSV (Hue-Saturation-Value)** uzayına çeviririz. **Hue (Renk Özü):** Rengin kendisidir (Sarı, Kırmızı). **Saturation (Doygunluk):** Rengin canlılığıdır (Soluk mu, cırtlak mı?). **Value (Parlaklık):** Işık miktarıdır. Böylece "Gölgede kalsa bile eğer rengin özü (Hue) kırmızıysa, onu bul" diyebiliriz.

B. Eşikleme (Binary Masking) Resmi siyah beyaza çevirme işlemidir. Girdi bir fotoğraftır. Algoritma der ki: "Eğer bir pikselin Hue değeri benim aralığımdaysa (örneğin 0 ile 20 arası) onu 1 (Beyaz) yap, değilse 0 (Siyah) yap". Çıktı, sadece ilgilendiğimiz nesnelerin beyaz leke olarak görüldüğü siyah bir ekrandır (Buna "Maske" denir).

C. Morfolojik İşlemler (Opening ve Closing) Elde ettiğimiz maske kusursuz değildir. Bazen nesnenin üstündeki bir parlama yüzünden ortasında siyah bir delik olabilir veya arka plandaki küçük bir toz beyaz görünebilir.

- **Opening (Açma):** Ufak tefek beyaz gürültüleri (tozları, yansımaları) silmek için kullanılır. Önce resmi aşındırır (Erosion), küçük lekeler yok olur, sonra ana nesneleri eski boyuna getirir (Dilation).
- **Closing (Kapama):** Nesnelerin içindeki delikleri kapatmak ve kopuk parçaları birleştirmek için kullanılır. Önce resmi şişirir (Dilation), delikler kapanır, sonra tekrar aşındırarak (Erosion) normal boyuta döner.

D. Sayım (Connected Component Labeling - CCL) Son olarak elimizde tertemiz beyaz lekeler var. CCL algoritması resmin sol üst köşesinden başlar, piksel piksel sağa ve aşağı tarar. Bir beyaz piksel gördüğünde, etrafındaki tüm beyaz piksellere "1" numarasını verir (Birinci nesne). Sonra taramaya devam eder, yeni bir beyaz piksel bulursa ona ve etrafındakilere "2" numarasını verir. En son kaç numaraya ulaştıysa, o kadar nesne var demektir. Ayrıca bu algoritma içinde çok küçük lekeleri (min_component_size) "nesne değil çöp" diyerek eleme mekanizmamız da vardır.

6. Önemli Fonksiyonlar / Sınıflar

- **color.py** içindeki **rgb_to_hsv(image)**:
 - Görevi: RGB görüntüyü NumPy matematiği kullanarak HSV'ye çevirmek.

- Önemi: Hazır kütüphane kullanmadan renk uzayı dönüşümü yapmak projenin en büyük mühendislik başarılarından biridir.

- **morphology.py** içindeki **opening()** ve **closing()**:

- Görevi: Siyah beyaz maskeyi temizlemek. Girdi olarak bir maske ve **kernel_size** (temizlik fırçasının boyutu) alır. Pikselleri komşularına göre kaydırarak pürüzleri yok eder.

- **counting.py** içindeki **extract_components(mask, min_size)**:

- Görevi: Temizlenmiş maske üzerindeki bağımsız adaları (nesneleri) bulmak ve etiketlemek. Çok küçük parçaları (**min_size** altındakileri) yoksayar.

- **pipeline.py** içindeki **run_pipeline_for_image()**:

- Görevi: Orkestra şefi. Resmi alır, renklere ayırır, temizler, saydırır ve sonucu bir paket (dictionary) halinde geri döndürür. Tüm parçaları birbirine bağlayan fonksiyondur.

7. Veri Akışı Örneği

1. **Girdi:** **data/real/raw/real_blue_5.png** (Mavi nesnelerin olduğu bir fotoğraf) yüklenir.
 2. **Dönüşüm:** Fotoğraf RGB'den HSV'ye dönüştürülür.
 3. **Eşikleme (Maskeleme):** **configs** dosyasında yazan "Mavi renk kurallarına" (Hue: 90-130) göre taranır ve sadece mavi olan yerler BEYAZ, gerisi SİYAH olan bir matris (Mask) üretilir.
 4. **Temizlik:** **opening** ile sahte beyaz noktalar silinir.
 5. **Sayım:** Bitişik beyaz pikseller gruplanır, boyutları **min_size: 100**'den büyük olanlar sayılır. Algoritma "6 nesne buldum" der.
 6. **Karşılaştırma:** **dataset.json** okunur. Orada da **expected_count: 6** yazmaktadır. Sistem hata payını 0 olarak belirler.
 7. **Çıktı:** Bulunan maske ve orijinalin üstüne çizilmiş (overlay) hali **.png** olarak **results** klasörüne kaydedilir.
-

8. Sunum İçin Anlatım Rehberi

Kısa Sunum Metni (Jüri Karşısında - 3 Dakika):

"Merhabalar, projemiz hazır bilgisayarlı görü kütüphanelerine (OpenCV gibi) bağımlı kalmadan, sıfırdan NumPy matris operasyonları ile geliştirdiğimiz bir renk segmentasyonu ve nesne sayma sistemidir.

Amacımız, karmaşık ışık koşulları altındaki gerçek dünya fotoğraflarında istenen renkteki objeleri tespit edip saymaktır. Bunun için sistemi sadece RGB ile değil, ışığa çok daha dayanıklı olan HSV renk uzayı ile tasarladık.

Sistemimiz bir görüntüyü alıyor, bizim belirlediğimiz eşik değerlerine (threshold) göre maskeliyor, ardından kendi yazdığımız 'Opening' ve 'Closing' gibi morfolojik filtrelerle gürültüleri temizliyor ve Connected Component algoritması ile geriye kalan nesneleri sayıyor.

En çok gurur duyduğumuz kısım, ince ayarlarımız sayesinde Kırmızı, Mavi, Yeşil ve Sarı renklerin tamamında kendi topladığımız gerçek dünya verilerinde %100 başarı oranına (accuracy) ulaşmış olmamızdır. Tüm bu süreci tam otomatik çalışacak ve otomatik rapor üretecek profesyonel bir mimariyle kurduk. Dinlediğiniz için teşekkür ederim."

9. Muhtemel Soru-Cevaplar

Soru: Neden OpenCV veya hazır bir araç yerine baştan (NumPy ile) yazdınız? **Cevap:** Amacımız sadece bir problemi çözmek değil, görüntü işleme algoritmalarının arka planda matematikle, piksellerle ve matrislerle nasıl çalıştığını içselleştirmektir. Kendi kernel tabanlı morfoloji algoritmamızı yazarak sistem üzerinde tam kontrol sağladık.

Soru: RGB varken neden HSV uzayını kullandınız? **Cevap:** RGB, ışık değişimlerine karşı çok zayıftır. Aynı cisim gölgede ve güneşte farklı RGB değerleri verir. HSV ise "Rengin kendisi" (Hue) ile "Işığı" (Value) birbirinden ayırır. Böylece karanlıkta kalan bir elmanın kırmızı olduğunu HSV ile çok daha rahat tespit edebiliriz. Bizim testlerimizde de HSV algoritmalarımız RGB algoritmalarımıza göre açık ara daha başarılı (%100) oldu.

Soru: Opening ve Closing nedir? Neden ihtiyaç duydunuz? **Cevap:** Renk eşikleme yaptığımızda görüntüde istemediğimiz gürültüler (noise) oluşuyor. Işık parlamaları yüzünden nesnelerin içinde siyah delikler, arka planda ise sahte beyaz noktalar beliriyor. Opening arkaplandaki beyaz tozları temizlerken, Closing nesne içindeki siyah delikleri kapatıp sistemi kusursuz bir sayıma hazırlar.

Soru: Birbirine çok yakın/bitişik nesneleri nasıl sayabildiniz? **Cevap:** Morfolojik temizleme sırasında uyguladığımız "Kernel" (Filtre çekirdeği) boyutunu çok ince (3x3 piksel) ayarladık. Ayrıca `min_component_size` gibi filtrelerle "nesne olmak için ne kadar büyük olmak gerektiğine" karar veren bir sistem kurduk.

10. Güçlü ve Zayıf Yönler

Güçlü Yönler:

- **Derinlemesine Anlayış:** Hazır kod kullanılmaması (No OpenCV).
- **Mükemmel Doğruluk:** Gerçek verilerdeki zorluklara rağmen HSV profillerinde ulaşılan %100 başarı.
- **Modülerlik:** Her renk ayarının dışarıdan bir JSON dosyasıyla (`multi-color-template.json`) kodla oynanmadan değiştirilebilmesi.

Zayıf Yönler / Eksikler:

- Sadece "renge" dayalı bir sistem. Eğer aranılan nesneyle arka plan aynı renkteyse sistem ayırt edemez.
- Işık yansımalarının çok ekstrem olduğu durumlarda eşikleme zorlaşır. (Deep Learning modelleri bu konuda daha iyidir).

Gelecekte Geliştirme:

- Canny Edge Detection (Kenar Bulma) gibi algoritmalar entegre edilerek sadece renge değil nesnenin şekline de bakılabilir.
- Farklı aydınlatma koşullarına otomatik adapte olan "Dinamik Eşikleme (Adaptive Thresholding)" eklenebilir.

11. Basit Özet (Hiç Bilmeyene Anlatım)

"Bizim yaptığımız program aslında dijital bir süzgeç. Elimizde karmaşık renklerin olduğu bir fotoğraf var ve biz sadece 'sarı' elmaları saymak istiyoruz. Program önce fotoğraftaki renkleri matematiksel bir dile çeviriyor. Sonra bizim ona verdiğimiz 'Sarı rengin şifresi şudur' kuralını uygulayarak fotoğrafın üzerine görünmez bir siyah kağıt seriyor. Sadece sarı olan yerleri delerek beyaz bırakıyor. Ortaya çıkan bu siyah beyaz haritada ufak tefek tozları matematiksel olarak bir silgi gibi siliyor. En son geriye kalan temiz beyaz adacıkları tek tek parmağıyla sayıp bize sonucu veriyor."